

Article

RAG-Based Natural Language Interface for Goal-Oriented Knowledge Graphs and Its Evaluation [†]

Kosuke Yano ^{1,*}, Yoshinobu Kitamura ^{2,*} and Kazuhiro Kuwabara ^{2,*} 

¹ Graduate School of Information Science and Engineering, Ritsumeikan University, Osaka 567-8570, Japan

² College of Information Science and Engineering, Ritsumeikan University, Osaka 567-8570, Japan

* Correspondence: is0584hf@ed.ritsumei.ac.jp (K.Y.); y-kita@fc.ritsumei.ac.jp (Y.K.); kuwabara@is.ritsumei.ac.jp (K.K.)

[†] This article is a revised and expanded version of: Yano, K.; Kitamura, Y.; Kuwabara, K. Natural Language Interface for Goal-Oriented Knowledge Graphs Using Retrieval-Augmented Generation. In Proceedings of the 17th International Conference on Agents and Artificial Intelligence (ICAART 2025), Porto, Portugal, 23–25 February 2025.

Abstract

Procedural knowledge is essential in specialized domains, and natural language tools for retrieving procedural knowledge are necessary for non-expert users to facilitate their understanding and learning. In this study, we focus on function decomposition trees, a framework for representing procedural knowledge, and propose a natural language interface leveraging Retrieval-Augmented Generation (RAG). The natural language interface converts the user's inputs into SPARQL queries, retrieving relevant data and subsequently presenting them in an accessible and chat-based format. Such a flexible and purpose-driven search facilitates users' understanding of functions of artifacts or human actions and their performance of these actions. We demonstrate that the tool effectively retrieves actions, goals, and dependencies using an illustrative real-world example of a function decomposition tree. In addition, we evaluated the system by comparing it with ChatGPT 4o and Microsoft GraphRAG. The results suggest that the system can deliver responses that are both necessary and sufficient for users' needs, while the outputs of other systems lack the key elements and return redundant information.

Keywords: function decomposition tree; retrieval-augmented generation; large language model

1. Introduction

In the context of knowledge management applications, effective handling of procedural knowledge is considered crucial [1]. Procedural knowledge refers to the processes and workflows that guide the execution of a specific task and is typically described in natural language through documents such as guidelines and manuals. For example, an official document regarding asset accounting procedures has been published by Japanese government in the field of accounting audits [2], and adherence to these procedures is required. Procedural knowledge can also be formalized and represented as structured knowledge using modeling languages such as BPMN [3] and IDEF3 [4]. To facilitate understanding and learning for non-expert users, a natural language interface is required for interacting with these forms of procedural knowledge.

Currently, natural language interfaces for extracting procedural knowledge are classified into three categories according to the nature of the knowledge sources provided to generate responses. The first approach relies solely on user queries as input to a general-purpose large language model (LLM). General-purpose LLMs are models that have not



Academic Editor: Charalampos Bratsas

Received: 4 November 2025

Revised: 18 December 2025

Accepted: 30 December 2025

Published: 7 January 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

been fine-tuned for a specific domain. They have the potential to implicitly capture procedural knowledge and generate step-by-step actions from high-level task descriptions, although the accuracy and consistency of generated procedures are limited [5]. The second approach provides the LLMs with domain-related procedural knowledge documents. In this approach, one method directly refers to textual documents, whereas the other indirectly refers to them through the derivation of a knowledge graph from the text [6]. The third approach directly provides the LLMs with a structured knowledge graph as the source of knowledge. In this approach, the knowledge graph provided to LLMs is manually constructed in advance. This last approach has been studied as part of a body of research into integrating LLMs with knowledge graphs [7].

In this study, we adopt the third approach. As a method of providing structured knowledge, we employ function decomposition trees, which are a descriptive method originally proposed for functional knowledge of artifacts in engineering design [8]. It has been extended to procedural knowledge of human actions called CHARM (Convincing Human Action Rationalized Model) in the health care field [9]. One of their important features is the explicit descriptions of the purposes or goals of actions. For example, in accounting auditing, the function decomposition tree has been used to model audit procedures based on auditing standards [10]. In this study, we propose a natural language interface based on function decomposition trees.

Many studies have explored systems that allow for searching knowledge graphs using natural language [11,12]. In the proposed interface, the schema of the function decomposition trees is first converted into RDF, a format supported by LLMs, to enable the model to refer to their data. Then, a SPARQL query is generated from the user's inputs to retrieve relevant information from the function decomposition trees, which is subsequently used to generate the final response [13].

To illustrate the efficacy and practical utility of the proposed method, the three approaches described above were compared in an evaluation experiment. The compared systems were (1) ChatGPT 4o, as a general-purpose LLM (the first approach), (2) Microsoft GraphRAG (v.2.4.0) [14], which indirectly refers to additional domain-related documents through the automatic derivation of a knowledge graph (the second approach), and (3) the proposed system, which utilizes a manually constructed knowledge graph (the third approach). For this evaluation, accounting auditing, a specialized domain that requires the use of procedural knowledge, was selected, and the system was evaluated through both quantitative metrics and expert-based assessments.

The remainder of this paper is structured as follows: Section 2 reviews the related work on knowledge graphs and natural language interfaces; Section 3 introduces the proposed method for developing an interface based on function decomposition trees; Section 4 presents a real-world example involving a function decomposition tree based on auditing procedures in accounting; Section 5 presents a comparative evaluation of the proposed method against other approaches, including Microsoft GraphRAG and a standard LLM; and Section 6 discusses the limitations of the proposed approach and outlines directions for future work. Finally, Section 7 concludes the paper.

2. Related Work

2.1. Function Decomposition Trees

A function decomposition tree provides a systematic framework for breaking down functions or actions into their constituent sub-functions or sub-actions, organizing them hierarchically in a tree-shaped knowledge graph [8,9]. Figure 1 illustrates an example of a function decomposition tree aimed at handling a cold.

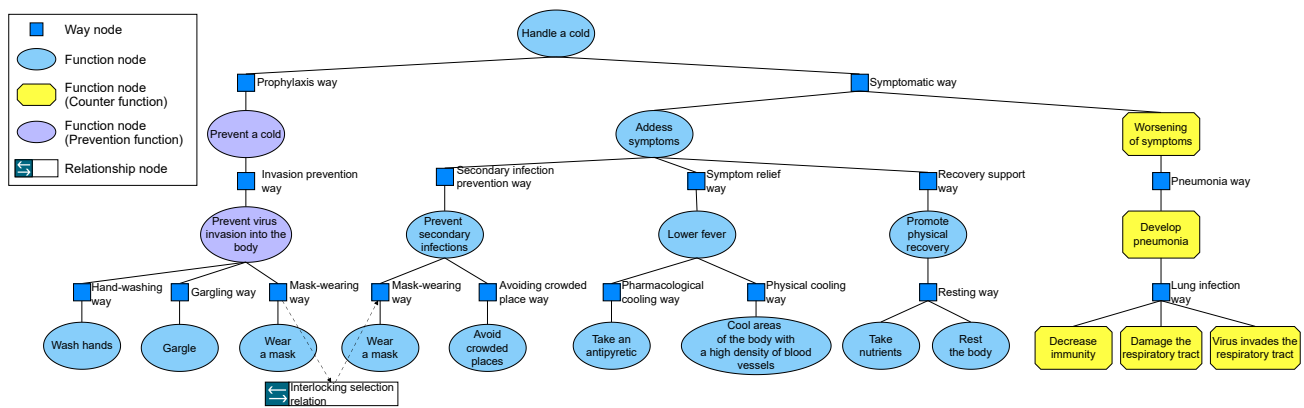


Figure 1. Example of a function decomposition tree for handling a cold.

The function decomposition tree includes three fundamental node types—*function nodes*, *way nodes*, and *relation nodes*—visualized as ellipses, squares, and arrow-linked rectangles, respectively. A *function node* denotes a *function* that can be accomplished either by artifacts or through human activities. Meanwhile, a *way node* expresses a *way of function/action achievement*, indicating the approach used to achieve a function or an action. Within the hierarchical tree, each *function node* can be expanded into multiple *way nodes*, which themselves decompose into further *function nodes*, revealing recursive layers of detailed action structures. As shown in Figure 1, “handling a cold” can be approached through either the “Prophylaxis way” or the “Symptomatic way”. This hierarchical structure effectively illustrates multiple possible *ways* toward achieving the same objective. In essence, the model captures the goal–method relationship underlying a series of actions.

Moreover, *function nodes* may include a *functional category* attribute that clarifies the specific nature or role of the function or action. For instance, in Figure 1, the action “Prevent virus invasion into the body” is categorized as a *prevention function* (depicted in purple), whereas the *counter functions* are shown in yellow.

Relation nodes express interdependencies among *way nodes*, indicating cases in which selecting one *way* influences other *ways*. For instance, in Figure 1, choosing the “mask-wearing way” to “prevent virus invasion into the body” also satisfies the action “prevent secondary infections”, as indicated by the relation node “interlocking selection relation.”

One of most important features of the function decomposition tree here is “goal-oriented”-ness. In comparison with other methods for process modeling such as BPMN [3] and IDEF3 [4], the goal of a function or an action and the *ways of achievement* are explicitly represented in a hierarchy. Although the importance of a “goal” in process representation has been pointed out and a goal-oriented extension of BPMN (GO-BPMN) has been proposed [15], the *ways of achievement* are implicit. In comparison with traditional function decomposition [16] and its extensions [17,18], the explicit description of the *ways of achievement* is a unique feature. Thus, users’ questions to the natural language interface developed in this research should include (1) What are the ways to achieve a goal? (2) What is the purpose of an action? (3) Which ways are impacted when applying a way?

2.2. Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) integrates LLMs with external retrieval mechanisms [19–22]. Conventional LLMs are prone to hallucinations and often struggle when dealing with queries beyond their training data, making it difficult to trace reasoning processes. RAG mitigates these issues by leveraging external data sources to enrich the generation process, thereby enhancing accuracy and reliability. In RAG, user queries trigger searches in an external database, and responses are generated based on the search results.

While most RAG systems employ vector databases as the external reference, recent research has focused on using knowledge graphs as knowledge sources [23]. Specifically, our approach focuses on retrieval from RDF-transformed function decomposition tree data, rather than on general knowledge graphs. This approach requires translating natural language queries into SPARQL.

Text-to-SPARQL conversion has been extensively studied [24–28]. However, prior research has mainly addressed general-purpose RDF datasets. In contrast, the present study concentrates on SPARQL generation over RDF data specifically designed to represent function decomposition trees, aiming at generating answers for the three types of user questions mentioned above.

As features of the function decomposition tree, functions or *ways* are described by labels, with `rdfs:label` having an important meaning in the converted RDF dataset. General text-to-SPARQL conversion tends to emphasize the structure of the knowledge graph and links between nodes rather than labels; it is difficult to extract important information contained in the labels of the converted RDF dataset. In the proposed system, we extend the RDF dataset by adding a keyword property as described later in Section 3.2 to perform keyword-based searches, and adopt the few-shot prompting approach to generate SPARQL queries instead of utilizing general text-to-SPARQL methods. This enables the system to specialize the query generation process for searching the function decomposition tree as the RDF data with the keyword property.

3. Methods

This section outlines the development of a natural language search interface for function decomposition tree data. To achieve this, we first identified the key conceptual components of the function decomposition tree and designed an RDF Schema tailored to them. Based on this schema, we implemented a conversion system that transforms the tree data into an RDF dataset. Finally, we developed a chat-based search interface that leverages RAG to enable efficient and flexible querying of the function decomposition tree.

3.1. Components in a Function Decomposition Tree

We define two fundamental components of a function decomposition tree: *functional categories* and *relations*. The former describe the types of functions or action nodes possess, while the latter express dependencies between *way nodes*.

Nodes that represent functions within a function decomposition tree are referred to as *Systemic Processes*, which are characterized by initial and end states, with the end state serving as the goal state. Such processes can be categorized into two subclasses based on their goals: *intended goal processes* and *unintended goal processes*. Our proposed approach incorporates both types of processes in defining and representing function decomposition trees.

3.1.1. Functional Category

Functional categories represent different types of functional or action nodes defined within the tree. Each functional category is treated as a specialization of function nodes. The *counter function* and the *prevention function* as described below are defined as subclasses of the *Systemic Process*.

3.1.2. Functional Category: Counter Function

A function decomposition tree may contain not only actions to be performed but also failure events and side effects resulting from these actions. These elements are categorized as *counter functions*. For instance, in Figure 1, the subtree rooted at the node labeled “worsening of symptoms”, shown in yellow, exemplifies this type of action. By explicitly representing failure events and side effects, the intentions behind corresponding actions

become clearer, potentially supporting defect prevention [8]. Similar to regular actions, shown in blue, failure or side-effect nodes are hierarchically decomposed—linking higher-level effects to lower-level causes. However, unlike normal actions, failure events or side effects are not intended to be executed. We define *counter functions* as a subclass of *unintended goal processes* within RDFS.

3.1.3. Functional Category: Prevention Function

Some actions in a function decomposition tree are preventive in nature and are thus classified as *prevention functions*. In Figure 1, the subtree labeled “prevent a cold” exemplifies such an action. Unlike typical actions that aim to achieve outcomes, preventive functions are designed to avert specific undesired states.

In the context of processes aimed at prevention, we can decompose the concept of *intended goal processes* into two categories: *actions* and *functions*. *Actions* refer to processes executed by living organisms, while *functions* are carried out by inanimate entities; both types can serve preventive purposes. Accordingly, we define *prevention actions* as a subclass of *actions*, and *prevention functions* as a subclass of *functions*. Since their overarching concepts of *actions* and *functions* are not explicitly distinguished within a function decomposition tree, *prevention actions* and *prevention functions* are treated as the same type of *functional category*. However, these concepts are defined as separate entities in an RDF to facilitate more precise querying.

3.1.4. Dependencies Between Ways

In a function decomposition tree, certain *ways* are interdependent in terms of their selection or activation. We classify these dependencies into two primary types: *interlocking selection relation* and *exclusive relation*. An *interlocking selection relation* indicates that selecting one *way* necessitates the selection of another. Conversely, the *exclusive relation* relationship ensures that if one *way* is selected, the other is precluded. Conversely again, the *exclusive relation* relationship enforces mutual exclusivity, ensuring that the selection of one *way* automatically excludes the other. *Exclusive relations* can be further divided into *sibling relations*, where the *ways* share the same parent node, and *contradictory relations*, where the dependent *ways* are located in distant branches of the tree. Both *interlocking selection relations* and *exclusive relations* are expressed through relationship nodes, defined as RDF properties that are sub-properties of their respective dependency categories. For instance, in Figure 1, the node connecting the two *mask-wearing ways* represents an *interlocking selection relation*. In the proposed method, we defined the dependencies between *ways* by treating relationship nodes as RDF properties. These nodes are described as sub-properties of each dependency type, allowing for a structured representation of the dependencies within the function decomposition tree.

3.1.5. Schema

Based on the components of a function decomposition tree defined in Section 3.1, we developed an RDF Schema (Figure 2) representing the structure of function decomposition trees. For simplification, the label and ID are described with identical content; thus, the label is omitted. Additionally, the descriptions of `rdfs:Class` and `rdf:Property` in some parts of Figure 2 are also partially omitted, as their semantics can be inferred through `rdfs:subClassOf` and `rdfs:subPropertyOf`.

3.2. Conversion to RDF Data

Based on the schema created in Section 3.1.5, we implemented a converter that transforms function decomposition tree data into RDF in Turtle format. In function decomposition trees, procedures and methods within actions are broken down into a tree structure,

with each node described in text form. In the converted RDF dataset, these textual descriptions were stored using the `rdfs:label` property, enabling text-based SPARQL queries.

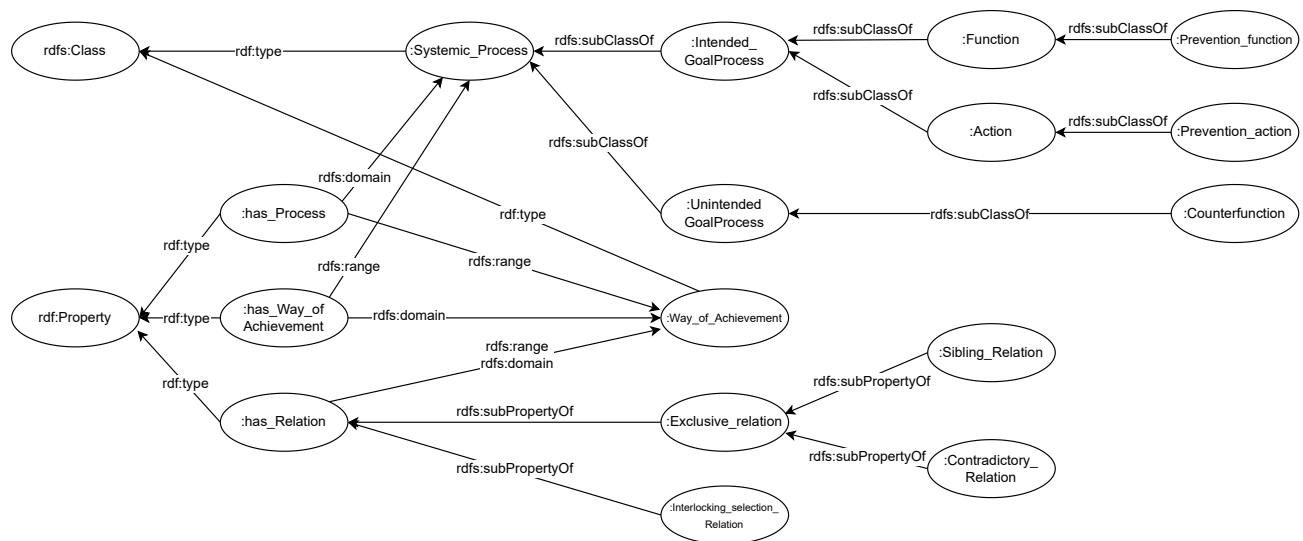


Figure 2. RDF Schema of a function decomposition tree.

As we described in Section 2, `rdfs:label` in the converted RDF dataset has an important meaning. However, because such queries rely on exact text matches, the presence of semantically equivalent but lexically different expressions could lead to search failures. To mitigate this issue, we introduced an additional `:keyword` property alongside `rdfs:label`. Keywords are automatically extracted from each node's `rdfs:label` using an LLM and stored under the `:keyword` property. This extension improves retrieval robustness by enabling semantic-level matching rather than strict lexical equality.

3.3. Searching for RDF Data Using a Chat-Based Interface with RAG

We implemented a chat-based interface that enables natural language search over function decomposition trees. User queries are first transformed into a SPARQL query through the application of an LLM. We employed OpenAI's gpt-4o-mini model for this transformation. The resulting query is executed on the RDF dataset, and the retrieved results are reformulated into natural language responses through the same model. As described in Section 2.1, we observed that user queries typically fall into three main categories, each handled by a specialized prompt configuration. The following subsections illustrate these three interaction modes using the function decomposition tree in Figure 1.

3.3.1. Decomposition of Actions and Ways

In the first pattern, the system identifies and presents specific actions or *ways* related to the user's query. This procedure begins with the extraction of keywords from a natural language sentence provided by the user. These keywords are then utilized to perform a filtering process on the `:keyword` property as described in Section 3.2. This filtering helps to narrow down the search to identify the node that is considered to be the closest match to the user's input. Next, a SPARQL query is generated to search for *function nodes* or *way nodes* that indicate the approach to achieving the identified node. The results from executing this query are then used to construct the response. The function of this SPARQL query is to identify sub-nodes that exist beneath a specific node within a function decomposition tree.

For example, when the user asks: What are the ways to address symptoms?, as illustrated in Figure 3, the extracted keywords "address" and "symptom" are used to identify relevant nodes. Subsequently, subordinate nodes under the matched node are identified and results

such as “Secondary infection prevention way,” “Recovery support way,” and “Symptom relief way” are returned. Finally, using these results, the system presents the following response to the user: To address symptoms, there are several ways you can employ: the “Secondary infection prevention way,” the “Recovery support way,” and the “Symptom relief way.” This approach enables the system to output actions related to the identified *ways* if the matched nodes are *way nodes*. Conversely, if the matched nodes are *action nodes*, the system can suggest *ways* to achieve specific actions.

What are the ways to address symptoms?

sample_rdf_bot

Generated SPARQL Query:

```

PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX ex: <https://example.org/>
SELECT ?obj_label
WHERE {
  {
    SELECT ?s (COUNT(?s) AS ?s_count)
    WHERE {
      ?s ex:keyword ?kwd .
      FILTER(contains(?kwd, "address") || contains(?kwd, "symptom")) .
    }
    GROUP BY ?s
    ORDER BY DESC(?s_count)
    LIMIT 1
  }
  ?s ex:has_Way_of_Achievement ?obj .
  ?obj rdfs:label ?obj_label .
}
```

sample_rdf_bot

Context:

obj_label: Secondary infection prevention way

obj_label: Recovery support way

obj_label: Symptom relief way

sample_rdf_bot

To address symptoms, there are several ways you can employ: the "Secondary infection prevention way," the "Recovery support way," and the "Symptom relief way."

Figure 3. Decomposing an action.

3.3.2. Identifying the Purpose of Actions

In the second pattern, the system infers and presents to the user a purpose or goal of an action or function associated with a specific action or *way* within the function decomposition tree. Similarly to as outlined in Section 3.3.1, after extracting keywords from the query, the system searches for function nodes located above the matched node in the decomposition tree. These keywords are then used to filter nodes via the `:keyword` property, enabling the system to identify the node that best matches the user’s input. Subsequently, a SPARQL query is generated to search for *function nodes* that are positioned above the specified node, which serves as the goal of either *function nodes* or *way nodes*. The results of executing this query are then used to construct the response. This SPARQL query is designed to locate action nodes that exist above a given node within the function decomposition tree.

Let us suppose a user inputs the following, as illustrated in Figure 4: What is the purpose of washing hands? The system extracts “wash” and “hand” as keywords and uses filtering to locate the relevant node. It then searches for action nodes above this node in the tree (Figure 1). The generated response is: The purpose of washing hands is to “Prevent virus invasion into the body.” The act of washing hands has a “prevention function.” This approach allows the system to output function nodes located above the node corresponding to the user’s input. Additionally, it can present the type of function that the function node aims to achieve.

What is the purpose of washing hands?

sample_rdf_bot

Generated SPARQL Query:

```

PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX ex: <https://example.org/>
SELECT ?obj_label ?obj_type_label
WHERE {
  {
    SELECT ?s (COUNT(?s) AS ?s_count)
    WHERE {
      ?s ex:keyword ?kwd .
      FILTER(contains(?kwd, "wash") || contains(?kwd, "hand")) .
    }
    GROUP BY ?s
    ORDER BY DESC(?s_count)
    LIMIT 1
  }
  {
    ?obj ex:has_Way_of_Achievement ?s .
  } UNION
  {
    ?p ex:has_Process ?s .
    ?obj ex:has_Way_of_Achievement ?p .
  }
  ?obj rdfs:label ?obj_label .
  ?obj rdf:type ?obj_type .
  ?obj_type rdfs:label ?obj_type_label
}
```

sample_rdf_bot

Context:
obj_label: Prevent virus invasion into the body,obj_type_label: Prevention Function

sample_rdf_bot

The purpose of washing hands is to "prevent virus invasion into the body." The act of washing hands has a "Prevention Function."

Figure 4. Searching for the purpose of actions.

3.3.3. Identifying Dependencies Between Ways

In the third pattern, the system identifies and presents dependencies among *ways* within the function decomposition tree to the user. The procedure begins with the extraction of keywords from a natural language sentence provided by the user, following procedures similar to those in Sections 3.3.1 and 3.3.2. These keywords are then used to filter nodes to identify the *way node* that best matches the user's input. Next, properties are searched for, where the node in question serves as the subject and another *way node* acts as the object. Finally, the results are utilized to generate a response. As defined in Section 3.1.5, a relation node is characterized by the `:has_Relation` property, which indicates *way nodes* as both the subject and predicate. Consequently, this SPARQL query is designed to search for relation nodes associated with a particular *way* within a function decomposition tree. For example, given the input: Which ways are impacted when applying the mask-wearing way?, as illustrated in Figure 5, the system then checks if the identified *way nodes* have any associated relational nodes. In this example, an "Interlocking selection" relation node exists, linked to the "Mask-wearing way." Using this connection, the system generates the response: The mask-wearing way has an "Interlocking selection relation" with the "mask-wearing way" to "Prevent secondary infections." Therefore, when applying the "Mask-wearing way", not only is the goal of "preventing virus invasion into the body" achieved, but also secondary infection prevention is significantly affected. This response clarifies that using the "mask-wearing way" for infection prevention also helps prevent secondary infections in others. In this pattern, the system provides the user with insights into additional outcomes that can be achieved when selecting a specific *way* to accomplish a particular action.

Which ways are impacted when applying the mask-wearing way?

sample_rdf_bot

Generated SPARQL Query:

```

PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX ex: <https://example.org/>
SELECT ?sp_label ?prop_type_label ?o_label ?op_label
WHERE {
  {
    SELECT ?s (COUNT(?s) AS ?s_count)
    WHERE {
      ?s ex:keyword ?kwd .
      FILTER(contains(?kwd, "mask") || contains(?kwd, "wear") || contains(?kwd, "way")) .
      ?s rdf:type ex:Way_of_Achievement .
    }
    GROUP BY ?s
    ORDER BY DESC(?s_count)
    LIMIT 1
  }
  ?sp ex:has_Way_of_Achievement ?s .
  ?sp rdfs:label ?sp_label .
  ?s ?prop ?o .
  ?o rdf:type ex:Way_of_Achievement .
  ?prop rdf:type ?prop_type .
  ?prop_type rdfs:label ?prop_type_label .
  ?o rdfs:label ?o_label .
  ?oParent ex:has_Way_of_Achievement ?o .
  ?oParent rdfs:label ?op_label
}

```

sample_rdf_bot

Context:
 sp_label: Prevent virus invasion into the body,prop_type_label:
 Interlocking_selection_Relation,o_label: Mask-wearing way,op_label: Prevent secondary
 infections

sample_rdf_bot

The mask-wearing way has an "Interlocking selection relation" with the "Mask-wearing way" for "Prevent secondary infections". Therefore, when applying the mask-wearing way, it not only achieves the goal of "preventing virus invasion into the body" but also significantly impacts the prevention of secondary infections.

Figure 5. Searching for the implication of selecting the way.

4. Real-World Example

To demonstrate the effectiveness of our approach, we present a real-world example involving a function decomposition tree about auditing procedures in accounting [10]. This function decomposition tree was constructed based on the official document *Asset Accounting Procedures* published by the Financial Services Agency of Japan [2]. The tree comprises 247 nodes, and the intricate interconnections among them make manual searching exceedingly difficult. Because the function decomposition tree is documented in Japanese, all searches were also conducted in Japanese. The following subsections illustrate examples of operations for the three patterns described in Section 3.3.

4.1. Decomposition of Actions and Ways in Accounting Audit

As an example of the first interaction pattern (“Decomposition of Actions and Ways”) described in Section 3.3.1, we demonstrate how the system searches for the ways to perform grouping in accounting audits (Figure 6). Here, *grouping* refers to the process of dividing assets into independent units.

Let us suppose that a user inputs the query: What are the ways for grouping assets? The system identifies the relevant keywords within the query, uses them to filter nodes that correspond to the user’s intent, and then examines the subnodes linked to these relevant nodes to generate an answer based on the retrieved results. In this example, the generated response is as follows: The ways to group assets include “when there is goodwill,” “when there is no goodwill with shared assets,” and “when there are shared assets.” Here, *shared assets* refer to assets that contribute to the cash flows of multiple asset groups or business units, while *goodwill* denotes the difference between the acquisition price and the fair value of net assets.

資産のグルーピングを行う方法は？
What are the ways for grouping assets?

sample_rdf_bot
Generated SPARQL Query:

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX ex: <https://example.org/>
SELECT ?obj_label
WHERE {
  SELECT ?s (COUNT(?s) AS ?s_count)
  WHERE {
    ?s ex:keyword ?kwd .
    FILTER(contains(?kwd, "資産") || contains(?kwd, "グルーピング") || contains(?kwd, "行う")) .
  }
  GROUP BY ?s
  ORDER BY DESC(?s_count)
  LIMIT 1
}
?s (ex:has_Way_of_Achievement | ex:has_Process) ?obj .
?obj rdfs:label ?obj_label .
```

Extracted keywords : "asset", "grouping", "perform"

sample_rdf_bot
Context:
obj_label: 共有資産とのれんの両方がない場合
obj_label: のれんがある場合
obj_label: 共有資産がある場合 :

sample_rdf_bot
資産のグルーピングを行う方法には、「共有資産とのれんの両方がない場合」、「のれんがある場合」、「共有資産がある場合」の3つの要素があります。
The following elements for grouping assets are: "when there are neither shared assets nor goodwill," "when there is goodwill," and "when there are shared assets."

共有資産とのれんの両方がない場合の方法は？
What are the ways when there are neither shared assets nor goodwill?

sample_rdf_bot
Generated SPARQL Query:

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX ex: <https://example.org/>
SELECT ?obj_label
WHERE {
  SELECT ?s (COUNT(?s) AS ?s_count)
  WHERE {
    ?s ex:keyword ?kwd .
    FILTER(contains(?kwd, "共有資産") || contains(?kwd, "のれん") || contains(?kwd, "ない")) .
  }
  GROUP BY ?s
  ORDER BY DESC(?s_count)
  LIMIT 1
}
?s (ex:has_Way_of_Achievement | ex:has_Process) ?obj .
?obj rdfs:label ?obj_label .
```

Extracted keywords : "shared asset", "goodwill", "absent"

sample_rdf_bot
Context:
obj_label: 他の資産又は資産グループのキャッシュ・フローから概ね独立したキャッシュ・フローを生み出す最小の単位で行う

sample_rdf_bot
共有資産とのれんの両方がない場合の方法は、「他の資産又は資産グループのキャッシュ・フローから概ね独立したキャッシュ・フローを生み出す最小の単位で行う」ことです。
When there are neither shared assets nor goodwill, the method is to "generate cash flows that are generally independent from the cash flows of other assets or asset groups, using the smallest unit possible."

Figure 6. Identifying the ways of grouping assets.

Users can refine their search through follow-up questions to the system. For instance, as shown in Figure 6, when a user inputs the query: what is the way when there is neither goodwill nor shared assets?, the generated response is: the way in case of neither goodwill nor shared assets is to “generate cash flows of other assets or asset groups at the smallest unit.”

In the reference document, this information is dispersed across multiple sections titled “Handling of Goodwill,” “Grouping of Assets”, and “Treatment of Shared Assets”, making it difficult to manually locate. By utilizing our system, users can efficiently access and view multiple *ways* of achieving the *grouping* in a unified manner.

4.2. Identifying Purpose of Actions in Accounting Audit

To show an example of the second interaction pattern (“Identifying Purpose of Actions”) described in Section 3.3.2, we demonstrate a query concerning the purpose of comparing current and projected future cash flows (Figure 7). Suppose a user inputs the query: What is the purpose of comparing the total amount of assets and the undiscounted future cash flows? The system extracts keywords from the query, filters the nodes that correspond to the user’s request, and then searches the superordinate nodes associated with those

matches to derive the purpose-related information. The generated response in this case is: The purpose of comparing the total amount of assets and the undiscounted future cash flows is “to recognize impairment losses.”

資産と割引前の将来キャッシュ・フローの総額を比較する目的は？

sample_rdf_bot
Generated SPARQL Query:

```

PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX ex: <https://example.org/>
SELECT ?obj_label ?s_type_label
WHERE {
  SELECT ?s (COUNT(?s) AS ?s_count)
  WHERE {
    ?s ex:keyword ?kwd .
    FILTER(contains(?kwd, "資産") || contains(?kwd, "割引前") || contains(?kwd, "将来キャッシュ・フロー") || contains(?kwd, "総額") || contains(?kwd, "比較")) .
  }
  GROUP BY ?s
  ORDER BY DESC(?s_count)
  LIMIT 1
}
UNION
{
  ?obj ex:has_Way_of_Achievement ?s .
  ?p ex:has_Process ?s .
  ?obj ex:has_Way_of_Achievement ?p .
}
?obj rdfs:label ?obj_label .
?s rdfs:type ?s_type_label .
?s_type rdfs:label ?s_type_label
}

```

What is the purpose of comparing the total amount of assets and the undiscounted future cash?

Extracted keywords : "asset", "comparing", "total amount", "undiscounted" and "future cash"

sample_rdf_bot
Context:
obj_label: 減損損失を認識する, s_type_label: SystemicProcess

sample_rdf_bot
資産と割引前の将来キャッシュ・フローの総額を比較する目的は「減損損失を認識する」ことです。
The purpose of comparing the total amount of assets and the undiscounted future cash flows is to "recognize impairment losses."

Figure 7. Identifying the purpose of comparing cash flows.

In the reference document [2], this information is presented in the form of procedural instructions, which makes it difficult for readers to infer the underlying objectives. By contrast, our system enables users to efficiently retrieve and understand the purposes behind such procedural actions.

4.3. Identifying Dependencies Between Ways in Accounting Audit

Finally, as an example of the third interaction pattern, “Identifying Dependencies between Ways,” introduced in Section 3.3.3, we show how the system identifies implications associated with selecting a specific *way* in the function decomposition tree (Figure 8). Consider the query: What are implications of selecting the book value allocation way? The system extracts keywords from the query, filters matching nodes, and examines whether the corresponding node is connected to any relation node. If the node that matches the user’s query has a *relation node*, the system searches for the node that is connected to the *relation node*. In this example, the generated answer is as follows: The book value allocation way has a “linked selection relationship” with the way of archiving the act of “determining the grouping way for shared assets”. By choosing this way, the grouping way for assets is determined.

In the reference document [2], dependencies between *ways* for each action are not explicitly described. Our system ensures these implicit relationships are explicit, allowing users to clearly understand how the selection of one *way* may influence others within the decomposition structure.

帳簿価額配分方式を選択すると、どのような影響がありますか？

What are the implications of selecting the book value allocation way?

sample_rdf_bot
Generated SPARQL Query:

```

PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX ex: <https://example.org/>
SELECT ?sp_label ?prop_type_label ?o_label ?op_label
WHERE {
  {
    SELECT ?s (COUNT(?s) AS ?s_count)
    WHERE {
      ?s ex:keyword ?kwd .
      FILTER(contains(?kwd, "帳簿価額") || contains(?kwd, "配分") || contains(?kwd, "方式")) .
      ?s rdf:type ex:Way_of_Achievement .
    }
    GROUP BY ?s
    ORDER BY DESC(?s_count)
    LIMIT 1
  }
  ?sp ex:has_Way_of_Achievement ?s .
  ?sp rdfs:label ?sp_label .
  ?s ?prop ?o .
  ?o rdf:type ex:Way_of_Achievement .
  ?prop rdf:type ?prop_type .
  ?prop_type rdfs:label ?prop_type_label .
  ?o rdfs:label ?o_label .
  ?oParent ex:has_Way_of_Achievement ?o .
  ?oParent rdfs:label ?op_label
}
  
```

Extracted keywords : "book value", "allocation", "way"

sample_rdf_bot

Context:
sp_label: グループ方法に応じて共用資産の比較方法を検討する, prop_type_label: 連動選択関係, o_label: 帳簿価額配分方式 (容認), op_label: 共用資産のグループ方法を選択する

sample_rdf_bot

「グループ方法に応じて共用資産の比較方法を検討する」行為の達成方式である帳簿価額配分方式 (容認) は「共用資産のグループ方法を決定する」行為の達成方式である「帳簿価額配分方式」と「連動選択関係」を持っています。帳簿価額配分方式を選択すると、「グループ方法に応じて共用資産の比較方法を検討する」だけでなく、「共用資産のグループ方法を決定する」ことも達成されます。

The method of book value allocation (approved), which is a means of achieving the action of considering the comparison method of shared assets according to the grouping method, has an interlocking selection relation with the method of determining the grouping method of shared assets. By selecting the book value allocation method, not only is the action of considering the comparison method of shared assets according to the grouping method achieved, but also the action of determining the grouping method of shared assets is achieved.

Figure 8. Identifying dependencies for the “book value allocation way”.

5. Comparison with Other Methods

5.1. Comparison Method and Result

As discussed in Section 1, there are three types of approaches to natural language interfaces for extracting procedural knowledge. To evaluate the proposed system, we compared the other two approaches with the proposed system. Microsoft GraphRAG was selected as a comparative approach as it is conceptually similar in that it utilizes knowledge graphs, deriving them from source documents and using them to generate responses [14]. To evaluate the performance of the system, we submitted similar questions to both Microsoft GraphRAG and a standard LLM (ChatGPT 4o). Their responses were then systematically compared. The system’s core and foundational pattern, involving the decomposition of actions and *ways*, was applied to the questions.

The questions posed concerned the grouping of assets in accounting auditing, in alignment with the discussion presented in Section 4.1. As we explained in Section 4.1, *grouping* refers to the process of dividing assets into independent units. In order to evaluate each response, an expert in accounting auditing evaluated the correctness and appropriateness of the responses; examined whether the necessary information was included; and compared the results.

5.1.1. The System Using the Function Decomposition Tree

As described in Section 4.1, the system answered that: The ways to group assets include “when there is goodwill,” “when there is no goodwill with shared assets,” and “when there are

shared assets.” This answer explicitly indicates that the *way* of grouping assets depends on the presence or absence of shared assets and goodwill.

5.1.2. Microsoft GraphRAG

To evaluate Microsoft GraphRAG, we provided the system with the same source document [2], and it autonomously constructed a knowledge graph based on the document’s content and subsequently generated responses by leveraging the graph structure (“local search”). The domain expert evaluated the system’s responses to the query: “What are the ways for grouping assets?” and provided two key observations. First, the response included extraneous content that was not directly relevant to the question. For instance, it referred to measuring impairment losses and considering remaining economic life. However, these topics are not directly related to the methods of asset grouping and were therefore deemed unnecessary to include in the response. Second, the response lacked key elements necessary for a comprehensive answer. While it addressed the presence of shared assets in the context of asset grouping, it failed to mention the treatment of goodwill, which is a critical consideration in grouping assets.

5.1.3. A Standard LLM

The domain expert evaluated ChatGPT’s response to the query: “What are the ways for grouping assets?” Similarly to Section 5.1.2, the response included content that was not directly relevant to the question. For example, statements concerning legal or contractual constraints do not have a direct impact on asset grouping and were therefore considered extraneous. In addition, certain statements in the response employed assertive language that may be misleading. For example, while the response states that assets should be grouped when they are physically or functionally related, there are cases in which such assets are not grouped, depending on the specific circumstances. Similarly to the responses discussed in Section 5.1.2, the LLM’s output also failed to address the treatment of goodwill, which is one of the key elements in asset grouping.

5.2. Results and Discussion

To quantitatively evaluate the systems, we prepared a gold-standard answer and computed the F1 score based on the gold-standard. The results are summarized in Table 1. The gold-standard answer to the query: “What are the ways for grouping assets?” focuses on the direct sub-actions and the *ways of achievement*. It consists of six items (shown as the “Number of total correct statements” in Table 1). Next, we counted the number of output statements generated by each system (shown as the “Number of output statements”) and classified each statement into one of three evaluation labels: *relevant*, *acceptable*, and *irrelevant*. Here, *relevant* indicates statements that correctly and directly correspond to one of the statements in the gold-standard answer; *acceptable* denotes statements that are partially correct or indirectly related; and *irrelevant* refers to statements that are either not directly relevant to the question or factually incorrect. Based on these labels, we computed strict precision, relaxed precision, recall, and F1 score. For strict precision, only *relevant* statements were regarded as correct, whereas for relaxed precision, both *relevant* and *acceptable* statements were considered correct.

One of the characteristics of a function decomposition tree is that it describes sub-actions and *ways* to achieve actions separately. Therefore, when decomposing an action, the proposed system first outputs the *ways* corresponding to the given action. In contrast, ChatGPT and GraphRAG directly output sub-actions as the result of the decomposition. To align the evaluation conditions, as shown in Figure 6, the proposed system was used to generate action decompositions through users’ follow-up questions, which elicited the actions contained in each *way*.

Table 1. F1 score of the systems for the query of decomposing the action.

	The Proposed System	ChatGPT-4o	Microsoft GraphRAG
Number of output statements	6	8	4
Number of correct statements outputted *	5	1 (acceptable: 4)	2
Strict precision	0.83	0.13	0.50
Relaxed precision	0.83	0.63	0.50
Recall	0.83	0.17	0.33
F1 Score	0.83	0.15 (0.27)	0.40

* The number of total correct statements is 6.

In addition, given the nature of the function decomposition tree as the underlying knowledge source, determining the appropriate level of granularity for decomposing actions is inherently difficult. To address this issue, we prepared such a gold-standard answer that focuses on the direct sub-actions of the given action and the *ways* of achieving actions.

As shown in Table 1, ChatGPT and GraphRAG achieve lower precision than the proposed system. As explained in Sections 5.1.2 and 5.1.3, the outputs of both Microsoft GraphRAG and ChatGPT contained irrelevant information that is not directly related to asset grouping methods. Such extraneous content may lead to confusion among users and hinder their understanding of the responses. In particular, ChatGPT shows a substantially poor strict precision, indicating that it has difficulty producing information that directly contributes to the achievement of the given action. In addition, ChatGPT and GraphRAG exhibited lower recall than the proposed system, suggesting that their responses lack important concepts, such as goodwill (as explained in Sections 5.1.2 and 5.1.3), required for a complete answer. In summary, by extracting information from the function decomposition tree, the proposed system generated answers that are both focused and explicitly specify the case classification.

In addition, we measured the response time from input to the completion of answer generation for each approach. The average response time (in seconds) of the proposed system, ChatGPT, and Microsoft GraphRAG are 9.4, 11.1, and 25.9, respectively. Microsoft GraphRAG also requires that a graph is constructed from the input documents before answer generation, and this graph construction process took 4 min and 5.7 s. While the proposed system runs substantially faster than GraphRAG, the comparison with ChatGPT shows greater variability due to server load, even though the average was computed over ten trials. The minimum response time of ChatGPT was 8.6 s, which is slightly faster than that of the proposed system (8.9 s). Therefore, the response times of the proposed system and ChatGPT can be considered comparable.

It is possible to improve the performance of both Microsoft GraphRAG and the standard LLM to some extent by carefully specifying prompts and incorporating relevant keywords into the queries. For example, in this case, the system may be queried with an input such as: “In an audit of impairment of fixed assets, explain how assets can be grouped in the presence of goodwill.” However, it would not be possible to include “goodwill” as part of the query input if the user does not have prior knowledge of the fact that the presence of goodwill affects case classification. Therefore, improving the quality of responses through prompt engineering requires a high level of domain expertise. This prerequisite may render the approach unsuitable for novice users, the expected users of the proposed system.

6. Limitations and Future Work

In Section 5, we focused on the “Decomposition of Actions and Ways” query pattern and conducted experiments on this pattern. This query type constitutes a critical component of the evaluation, as it requires exhaustive responses that fully cover multiple *ways* of action decomposition. Among the three query patterns described in Section 3.3, the “Identifying Purpose of Actions” and “Identifying Dependencies between Ways” patterns were not examined experimentally in this study. A systematic evaluation of these two patterns lies outside the scope of the present work and will be covered in future work.

As shown in Table 1, the proposed system does not achieve perfect accuracy under the existing evaluation framework, indicating certain limitations of the current approach. The accuracy of the system’s output is determined by two essential components: “identification of the query pattern” and “localization of the relevant node.” In the former, the system demonstrates robust performance in determining the appropriate query type. For example, the system can identify: What principles determine the boundaries of asset groups? as “Decomposition of Action and Ways”, which includes abstract phrasing. In contrast, localization of the relevant node remains challenging, particularly for queries that deviate from the idealized form of queries used in Section 5. Since the system relies on keyword extraction, the system may fail to associate a query with the correct node when the query employs semantic equivalents, contains conceptual ambiguity, or is excessively short and thus provides only a limited number of available keywords. As future work, we plan to implement semantic retrieval methods, such as embedding-based searches, which may allow the system to retrieve semantically related nodes even when the exact terminology does not appear in the graph. This extension will enable the system to respond more flexibly to diverse query expressions.

In addition, the *functional categories* defined in Section 3.1 are currently used only for presentation purposes and are not yet fully integrated into the search mechanism. In future work, we aim to enhance the search process by incorporating these functional categories more robustly. For example, if a user query is related to “preventive measures”, the system could insert a corresponding FILTER into the SPARQL query to restrict results to nodes labeled with *Prevention function*.

A user study can be considered to evaluate the proposed system. As a concrete example of system application, we will recruit novice students and divide them into two groups. One group will use the proposed system and the other group will use ChatGPT to solve tasks that require specialized knowledge. Then, we will analyze their performance scores and assess their subjective experiences through questionnaires.

7. Conclusions

In this paper, we propose a system that provides users with goal-oriented knowledge through a natural language interface. Using an RDF-converted function decomposition tree, users can retrieve (1) finer-grained functions/actions, (2) goals/purposes, or (3) dependencies of specified functions or actions. Such knowledge facilitates users’ understanding of functions of artifacts or human actions and their performance of these actions. The functional categories of the function decomposition tree, such as *prevention functions* and *counter functions*, particularly clarify the teleological roles of the functions.

As shown in Section 5, we evaluated the core pattern, retrieving functions/actions. Compared with other approaches, the proposed system provided only essential, directly relevant knowledge. As reflected by the higher quantitative scores shown in Table 1, these results demonstrate that our system can deliver responses that are both necessary and sufficient for users’ needs.

In the structured approach to procedural knowledge extraction as proposed in this paper, information is obtained from domain experts to construct the model, which increases the cost but ensures accuracy. In unstructured methods such as ChatGPT and GraphRAG, as we discussed in Section 1, unstructured natural language is the input to the system, and necessary information is extracted based on the user's query and accompanying prompt. In contrast to the structured approach, relying on unstructured methods reduces the cost but introduces limitations, including hallucinations and no guarantee of accuracy, as we demonstrated in Section 5. Overall, there is a trade-off between accuracy and cost, and the structured approach is particularly effective in domains where accuracy is critical, such as for accounting audits. This study highlights the potential of structured knowledge representations as a complementary alternative to unstructured language models, especially in settings where reliability and user trust are paramount.

Author Contributions: Conceptualization, K.Y.; methodology, K.Y. and K.K.; software, K.Y. and K.K.; validation, K.Y.; investigation, K.Y.; writing—original draft preparation, K.Y.; writing—review and editing, Y.K. and K.K.; visualization, K.Y.; supervision, Y.K. and K.K.; project administration, Y.K. and K.K.; funding acquisition, K.K. All authors have read and agreed to the published version of the manuscript.

Funding: This work was partially supported by JSPS KAKENHI Grant Number JP24K15078.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data presented in this study are available on request from the corresponding authors due to legal and ethical reasons.

Acknowledgments: The authors would like to thank Hiroshi Taki, Ritsumeikan University, for providing valuable comments and evaluating accounting audits.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Celino, I.; Carriero, V.A.; Azzini, A.; Baroni, I.; Scrocca, M. Procedural knowledge management in Industry 5.0: Challenges and opportunities for knowledge graphs. *J. Web Semant.* **2024**, *84*, 100850. [CrossRef]
2. Financial Services Agency. Opinion on Setting Accounting Standards for Impairment of Fixed Assets. 2002. Available online: <https://www.fsa.go.jp/news/newsj/14/singi/f-20020809-1/f-20020809c.pdf> (accessed on 30 October 2025).
3. Chinosi, M.; Trombetta, A. BPMN: An introduction to the standard. *Comput. Stand. Interfaces* **2012**, *34*, 124–134. [CrossRef]
4. Mayer, R.J.; Menzel, C.P.; Painter, M.K.; deWitte, P.S.; Blinn, T.; Perakath, B. *Information Integration for Concurrent Engineering (IICE): IDEF3 Process Description Capture Method Report*; Technical Report KBSI-IICE-90-TR-01; United States Air Force, Armstrong Laboratory, Human Resources Directorate, Wright-Patterson Air Force Base: Dayton, OH, USA, 1992.
5. Huang, W.; Abbeel, P.; Pathak, D.; Mordatch, I. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In Proceedings of the 39th International Conference on Machine Learning, Baltimore, MD, USA, 17–23 July 2022; pp. 9118–9147.
6. Carriero, V.A.; Azzini, A.; Baroni, I.; Scrocca, M.; Celino, I. Human evaluation of procedural knowledge graph extraction from text with large language models. In Proceedings of the International Conference on Knowledge Engineering and Knowledge Management, Amsterdam, The Netherlands, 26–28 November 2024; Springer: Berlin, Germany, 2024; pp. 434–452.
7. Ibrahim, N.; Aboulela, S.; Ibrahim, A.; Kashef, R. A survey on augmenting knowledge graphs (KGs) with large language models (LLMs): Models, evaluation metrics, benchmarks, and challenges. *Discov. Artif. Intell.* **2024**, *4*, 76. [CrossRef]
8. Kitamura, Y.; Kashiwase, M.; Fuse, M.; Mizoguchi, R. Deployment of an Ontological Framework of Functional Design Knowledge. *Adv. Eng. Inform.* **2004**, *18*, 115–127. [CrossRef]
9. Nishimura, S.; Kitamura, Y.; Sasajima, M.; Williamson, A.; Kinoshita, C.; Hirao, A.; Hattori, K.; Mizoguchi, R. CHARM as activity model to share knowledge and transmit procedural knowledge and its application to nursing guidelines integration. *J. Adv. Comput. Intell.* **2013**, *17*, 208–220. [CrossRef]

10. Taki, H.; Sogawa, S.; Mura, K.; Kitamura, Y. Ontological Approach for Modeling of a Financial Statement Audit—A Model of Actions of an Audit of Impairment of Fixed Assets. In Proceedings of the 37th Annual Conference of the Japanese Society for Artificial Intelligence, Kumamoto, Japan, 6–9 June 2023; The Japanese Society for Artificial Intelligence: Tokio, Japan, 2023; p. 2L6GS305.
11. Damjanovic, D.; Agatonovic, M.; Cunningham, H. FREyA: An interactive way of querying Linked Data using natural language. In Proceedings of the Semantic Web, ESWC 2011 Workshops: ESWC 2011 Workshops, Heraklion, Greece, 29–30 May 2011; Revised Selected Papers 8; Springer: Berlin/Heidelberg, Germany, 2012; pp. 125–138.
12. Ochienig, P. PAROT: Translating natural language to SPARQL. *Expert Syst. Appl.* **2020**, *5*, 100024. [\[CrossRef\]](#)
13. Yano, K.; Kitamura, Y.; Kuwabara, K. Natural Language Interface for Goal-Oriented Knowledge Graphs Using Retrieval-Augmented Generation. In Proceedings of the 17th International Conference on Agents and Artificial Intelligence (ICAART 2025), Porto, Portugal, 23–25 February 2025; SCITEPRESS: Setúbal, Portugal, 2025; Volume 3, pp. 975–982. [\[CrossRef\]](#)
14. Edge, D.; Trinh, H.; Cheng, N.; Bradley, J.; Chao, A.; Mody, A.; Truitt, S.; Metropolitansky, D.; Ness, R.O.; Larson, J. From local to global: A graph rag approach to query-focused summarization. *arXiv* **2024**, arXiv:2404.16130. [\[CrossRef\]](#)
15. Greenwood, D.; Ghizzioli, R. Goal-Oriented Autonomic Business Process Modelling and Execution. In *Multiagent Systems*; Ahmed, S., Karsiti, M.N., Eds.; IntechOpen: London, UK, 2009; Chapter 3. [\[CrossRef\]](#)
16. Pahl, G.; Beitz, W.; Feldhusen, J.; Grote, K. *Engineering Design: A Systematic Approach*; Springer: London, UK, 2006.
17. Malmqvist, J. Improved function-means trees by inclusion of design history information. *J. Eng. Des.* **1997**, *8*, 107–117. [\[CrossRef\]](#)
18. Johnson, S.B. Goal-function tree modeling for systems engineering and fault management. In Proceedings of the AIAA Infotech@Aerospace (I@A) Conference, Boston, MA, USA, 19–22 August 2013; p. 4576.
19. Gao, Y.; Xiong, Y.; Gao, X.; Jia, K.; Pan, J.; Bi, Y.; Dai, Y.; Sun, J.; Wang, H. Retrieval-augmented generation for large language models: A survey. *arXiv* **2023**, arXiv:2312.10997.
20. Yu, H.; Gan, A.; Zhang, K.; Tong, S.; Liu, Q.; Liu, Z. Evaluation of Retrieval-Augmented Generation: A Survey. In *Communications in Computer and Information Science*; Zhu, W., Xiong, H., Cheng, X., Cui, L., Dou, Z., Dong, J., Pang, S., Wang, L., Kong, L., Chen, Z., Eds.; Springer: Singapore, 2025; pp. 102–120.
21. Fan, W.; Ding, Y.; Ning, L.; Wang, S.; Li, H.; Yin, D.; Chua, T.S.; Li, Q. A survey on RAG meeting LLMs: Towards retrieval-augmented large language models. In Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Barcelona, Spain, 25–29 August 2024; pp. 6491–6501.
22. Cheng, M.; Luo, Y.; Ouyang, J.; Liu, Q.; Liu, H.; Li, L.; Yu, S.; Zhang, B.; Cao, J.; Ma, J.; et al. A Survey on Knowledge-Oriented Retrieval-Augmented Generation. *arXiv* **2025**, arXiv:2503.10677.
23. Procko, T.T.; Ochoa, O. Graph Retrieval-Augmented Generation for Large Language Models: A Survey. In Proceedings of the 2024 Conference on AI, Science, Engineering, and Technology (AIxSET), Laguna Hills, CA, USA, 30 September–2 October 2024; pp. 166–169. [\[CrossRef\]](#)
24. Kovriguina, L.; Teucher, R.; Radyush, D.; Mouromtsev, D.; Keshan, N.; Neumaier, S.; Gentile, A.; Vahdati, S. SPARQLGEN: One-Shot Prompt-based Approach for SPARQL Query Generation. In Proceedings of the SEMANTICS 2023 EU: 19th International Conference on Semantic Systems, Leipzig, Germany, 20–22 September 2023.
25. Avila, C.V.S.; Vidal, V.M.; Franco, W.; Casanova, M.A. Experiments with text-to-SPARQL based on ChatGPT. In Proceedings of the 2024 IEEE 18th International Conference on Semantic Computing (ICSC), Laguna Hills, CA, USA, 5–7 February 2024; pp. 277–284.
26. Markowitz, E.; Ramakrishna, A.; Dhamala, J.; Mehrabi, N.; Peris, C.; Gupta, R.; Chang, K.W.; Galstyan, A. Tree-of-Traversals: A Zero-Shot Reasoning Algorithm for Augmenting Black-box Language Models with Knowledge Graphs. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Bangkok, Thailand, 11–16 August 2024; pp. 12302–12319.
27. Zahera, H.M.; Ali, M.; Sherif, M.A.; Moussallem, D.; Ngomo, A.N. Generating SPARQL from Natural Language Using Chain-of-Thoughts Prompting. In Proceedings of the SEMANTICS, Amsterdam, The Netherlands, 17–19 September 2024; pp. 353–368.
28. Hernandez-Camero, I.V.; Garcia-Lopez, E.; Garcia-Cabot, A.; Caro-Alvaro, S. Context-Aware Few-Shot Learning SPARQL Query Generation from Natural Language on an Aviation Knowledge Graph. *Mach. Learn. Knowl. Extr.* **2025**, *7*, 52. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.